

1FA018: Exercise set 3

Leandro Morita, Master student

December 2025

The font family used in this document is Palatino.

The code for the solutions made on this document are also placed on a GitHub repository ¹

Question 1: Systematic uncertainties

A PhD student measures the height of a table-top Xmas tree with a steel ruler and finds it to be 99.6 cm. The space between the markers of the ruler is 1 mm. The steel ruler is calibrated at room temperature (20°C). However, the student's office is at Ångström, which is considerably colder (16°C) since they have to work with the window open due to bad ventilation. The student looks up the expansion coefficient of steel in a table and finds it to be $0.0005 \pm 0.0001^\circ\text{C}^{-1}$.

a) What is the height of the tree if the student corrects for the thermal expansion of the steel ruler?

As $\alpha = 0.0005 [\frac{\text{cm}}{\text{cm} \cdot ^\circ\text{C}}]$, $T_{\text{actual}} = 16 [^\circ\text{C}]$, $T_{\text{calibrated}} = 20 [^\circ\text{C}]$ and $L_{\text{measured}} = 99.6 [\text{cm}]$

$$\Delta L = \alpha(T_{\text{actual}} - T_{\text{calibrated}})L_{\text{measured}}$$

$$\Delta L = -0.1992 \text{ cm}$$

Then, the corrected tree height would be

$$L' = L_{\text{measured}} + \Delta L = 99.4008 \text{ cm}$$

¹<https://github.com/Lemorita95/1FA018>

b) Calculate the total systematic uncertainty based on the sources which have been mentioned directly or indirectly.

The sources of uncertainty are: the ruler grading and the expansion coefficient.

$$\sigma_{grading} = 1 \text{ mm} = 0.1 \text{ cm}$$
$$\sigma_{\alpha} = 0.0001 \frac{\text{cm}}{\text{cm} \cdot ^\circ\text{C}} \cdot |16 - 20| ^\circ\text{C} \cdot 99.6 \text{ cm} = 0.0398 \text{ cm}$$

Assuming full correlation ($\rho = 1$, conservative), the uncertainties will be added linearly.

$$\sigma_{total} = \sigma_{grading} + \sigma_{\alpha} = 0.1398 \text{ cm}$$

c) Can you think of some other source of systematic uncertainty that we cannot quantify with the given information?

The room temperature (16°C).

d) So far, we have focused on the precision of the Xmas tree height. Describe a situation where we would be interested in the tolerance, rather than the precision?

When choosing the table / tree, we are interested in the tolerance in mass (will the table withstand the tree?) and height (what is the maximum height of the tree/table so it fit our room ceiling).

Question 2: Monte Carlo methods

Implement a function to transform a uniform distribution into a Gaussian with mean $\mu = 5.0$ with a width $\sigma = 2.0$. Plot distributions for u , v as well as for x_1 , x_2 . Check that the generated distribution has the desired mean and width, using for example a simple LSQ fit.

To begin this problem, define the random number generator *RNG* in python with `SeedSequence([42,2])` and `default_rng()`. Then, define a function that takes as arguments the number of samples N , the desired mean μ , the desired width σ and the *RNG* for random number generation and reproducibility. The implementation is shown in Figure 1 and the result distributions in Figure 2.

We then proceed to check if the generated distribution, for this exercise we choose x_1 , has the desired mean and width, this will be done by a

```
def polar_method(N, mu, sigma, rng):
    i=0
    while i < N:
        u = rng.uniform(-1, 1) # draw one sample
        v = rng.uniform(-1, 1) # draw one sample

        r = u**2 + v**2

        if r > 1 or r == 0:
            continue

        m = np.sqrt(-2*np.log(r)/r)
        x1 = mu + sigma * u * m
        x2 = mu + sigma * v * m

        i += 1
        yield u, v, x1, x2
```

Figure 1: Polar method implementation.

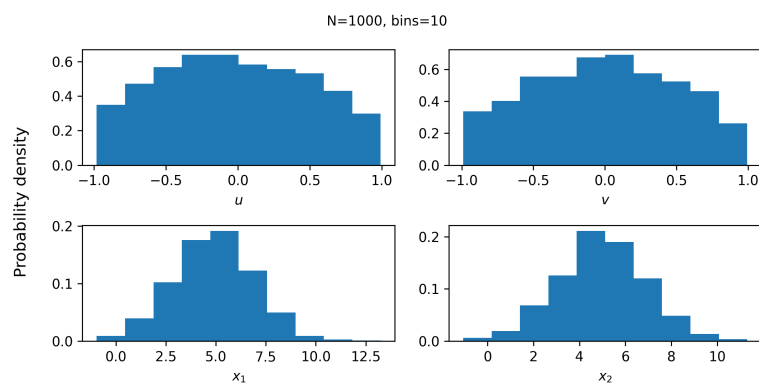


Figure 2: Polar method results.

```
def residuals(parameters, bin_edges, hist_count, var_count, func_target):
    """
    chi square residuals for generalized least squares
    """
    mu, sigma = parameters

    delta = hist_count - func_target(bin_edges, mu, sigma, hist_count.sum())

    G = np.linalg.cholesky(var_count)
    r = np.linalg.solve(G, delta)

    return r
```

Figure 3: χ^2 residual function.

```
def func_target(bin_edges, mu, sigma, N):
    """
    this function computes the expected value of a binned normal distribution
    method: compute the expected bin value of x by the CDF of x_i+1 and x_i
    """
    cdf = norm.cdf
    delta = cdf(bin_edges[1:], mu, sigma) - cdf(bin_edges[:-1], mu, sigma)
    return N * delta
```

Figure 4: Gaussian distribution for number of events.

generalized least squares fit with χ^2 residuals, so it can be reused for the further fitting in this exercise set as it accepts both diagonal and full covariance matrices.

First we bin the data to get the counting of events. The covariance of x_1 assumes a Poisson approximation, where the variance of each bin is the number of events in the bin. Then, our goal with the LSQ fit is to find the best Gaussian distribution (minimum χ^2), that fits our data. This is implemented in python using the functions shown in Figures 3 and 4.

Using `scipy.optimize.least_squares()` with the previous residual function yields the following parameter estimation, also shown in Figure 5.

$$\hat{\mu}_{\chi^2} = 4.8614$$

$$\hat{\sigma}_{\chi^2} = 1.9975$$

This result is for $N = 1000$. It was noted that when increasing N , $\mu_{\chi^2} \rightarrow \mu = 5.0$ and $\sigma_{\chi^2} \rightarrow \sigma = 2.0$.

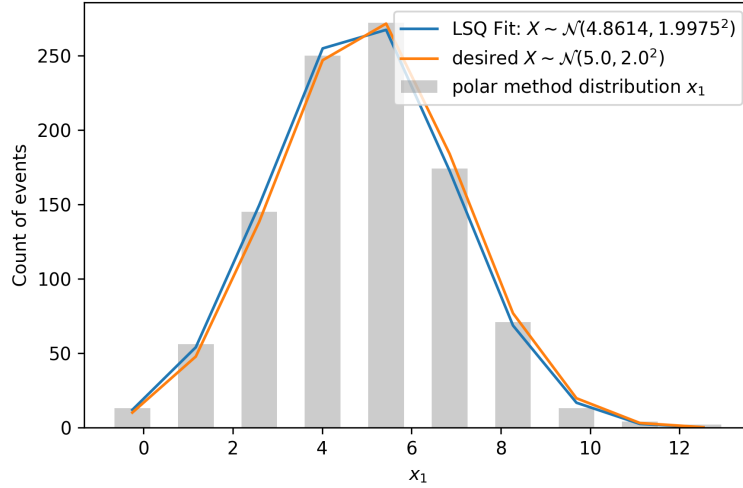


Figure 5: Generated function using polar method.

[0.585	0.225	0.09	0.	0.	0.	0.	0.	0.	0.]
[0.225	0.36	0.225	0.09	0.	0.	0.	0.	0.	0.]
[0.09	0.18	0.36	0.18	0.09	0.	0.	0.	0.	0.]
[0.	0.09	0.18	0.36	0.18	0.09	0.	0.	0.	0.]
[0.	0.	0.09	0.18	0.36	0.18	0.09	0.	0.	0.]
[0.	0.	0.	0.09	0.18	0.36	0.18	0.09	0.	0.]
[0.	0.	0.	0.	0.09	0.18	0.36	0.18	0.09	0.]
[0.	0.	0.	0.	0.	0.09	0.18	0.36	0.18	0.09]
[0.	0.	0.	0.	0.	0.	0.09	0.225	0.36	0.225]
[0.	0.	0.	0.	0.	0.	0.	0.09	0.225	0.585]]

Figure 6: Response matrix R.

Question 3: Unfolding

Let $m = n = 10$. Take the function you implemented in exercise 2 for the Gaussian of mean $\mu = 5.0$ with a width $\sigma = 2.0$.

a) **Generate a distribution with 1000 events with these properties, organize them in a histogram with 10 bins and “smear” it with the response matrix R: $f'(x') = Rf(x)$. Extract the parameters of the folded distribution (mean and width), by e.g. a LSQ fit.**

Define a random number generator *RNG* in python with a *new* Seed-Sequence([42,3]).

Fill in the remaining elements of the Response matrix R. Figure 6.

Generate and bin data, similarly to what was done for the previous question. The covariance of $f(x)$, V_f is diagonal and equal to the count of events in each bin, Poisson approximation.

$$V_f = \begin{bmatrix} \sigma_{1,1}^2 & 0 & \cdots & 0 \\ 0 & \sigma_{2,2}^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_{10,10}^2 \end{bmatrix} \quad (1)$$

Fold the data using the R matrix, Equations 2 and 3

$$f'(x') = R f(x) \quad (2)$$

For the covariance,

$$V_{f'} = E[(f'(x') - E[f'(x')])^2]$$

in matrix notation,

$$V_{f'} = E[(f'(x') - E[f'(x')]) \cdot (f'(x') - E[f'(x')])^T]$$

Using Equation 2 and

$$E[f'(x')] = R.E[f(x)]$$

yield the covariance in matrix form:

$$V_{f'} = R V_f R^T \quad (3)$$

Apply the same LSQ fit process from before in at the resulting matrix of Equations 2 and 3. Note that $V_{f'}$ is not diagonal, since the response matrix (Figure 6), introduces bin migration. Then, the estimated parameters of $f'(x')$:

$$\hat{\mu}_{f'} = 4.9774 \quad (4)$$

$$\hat{\sigma}_{f'} = 2.5802 \quad (5)$$

Figure 7 shows the original distribution $f(x)$, the folded distribution $f'(x')$ and the LSQ fit on the folded distribution. In figure 7 we can see the widening and reduction of the number of samples causes by the response matrix.

b) Generate 1000 new events of the variable x' , using the parameters extracted from the folded histogram in 3a). Now use correction factor

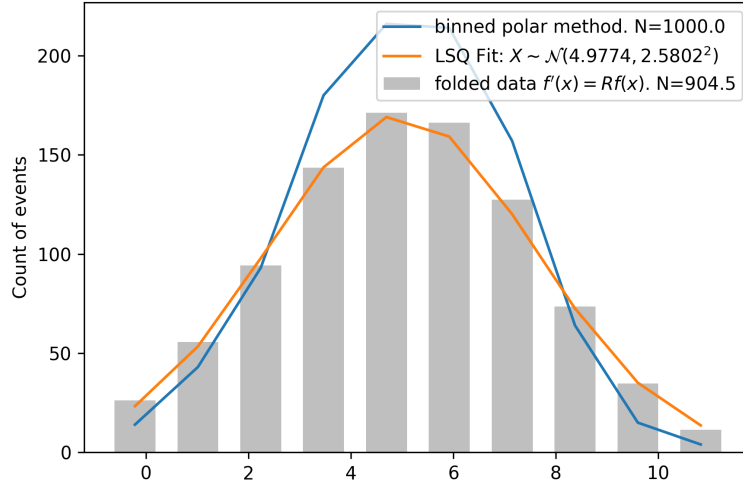


Figure 7: Folded distribution.

method to unfold this distribution to obtain an estimate of “the truth”, i.e. $f(x)$. Please also calculate the uncertainties (covariance matrix). Illustrate with histograms/plots on all levels of generating and unfolding.

Start by generating a new distribution with $N = 1000$ events and parameters $\mu = \hat{\mu}_{f'} = 4.9774$ and $\sigma = \hat{\sigma}_{f'} = 2.5802$. Use the same seed sequence as in 3.a). Choose one of the distributions produced by the polar method, Figure 8.

Proceed to bin the distribution. Since this new distribution is for a new random variable, it was assumed independence on the events and applied the Poisson approximation (similar to Equation 1). The covariance $V_{f'(x')}$ is a vector (or diagonal matrix) with values equal to the number of events in the bin.

In order to maintain the alignment of the new distributions to apply the correction factor, the same bin edges of Figure 7 were applied to the binning of this new distribution.

The left chart of Figure 9 shows the binned distribution, note that due to the selected bin edges, there is sample loss.

Next, calculate the correction factor C_i from Figure 7 using Equation 6, where $N_{f(x_i)}$ and $N_{f'(x'_i)}$ are the number of events in the i -bin of the polar method (blue line) and folded data (grey bar) obtained in question 3.a), respectively. The correction factor values are shown in the right chart of

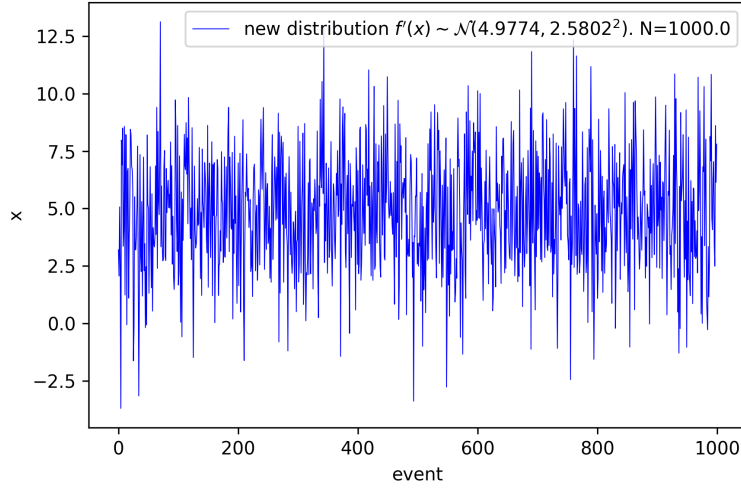


Figure 8: New gaussian distribution given estimated parameters of folded distribution.

Figure 9.

$$C_i = \frac{N_{f(x_i)}}{N_{f'(x'_i)}} \quad (6)$$

Proceed to compute the unfolded distribution with $C = (C_1, C_2, \dots, C_{10})$

$$f(x) = C f'(x')$$

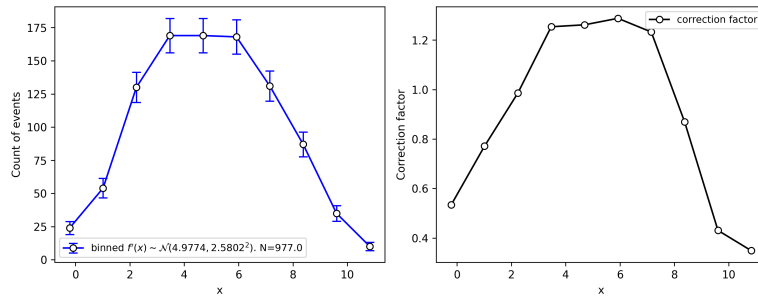


Figure 9: Binned distribution and Correction factor.

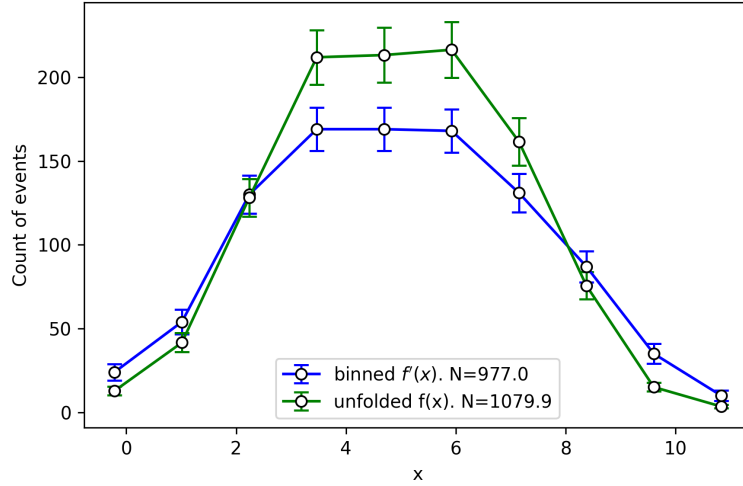


Figure 10: Unfolded distribution.

[	6.834	0.	0.	0.	0.	0.	0.	0.	0.	0.]
[	0.	32.119	0.	0.	0.	0.	0.	0.	0.	0.]
[	0.	0.	126.387	0.	0.	0.	0.	0.	0.	0.]
[	0.	0.	0.	265.721	0.	0.	0.	0.	0.	0.]
[	0.	0.	0.	0.	269.084	0.	0.	0.	0.	0.]
[	0.	0.	0.	0.	0.	278.733	0.	0.	0.	0.]
[	0.	0.	0.	0.	0.	0.	199.101	0.	0.	0.]
[	0.	0.	0.	0.	0.	0.	0.	65.749	0.	0.]
[	0.	0.	0.	0.	0.	0.	0.	0.	6.491	0.]
[	0.	0.	0.	0.	0.	0.	0.	0.	0.	1.215]]

Figure 11: Covariance matrix of unfolded distribution.

and

$$V_{f(x)} = C^2 V_{f'(x')}$$

$V_{f(x)}$ is a vector (or diagonal matrix) due to the poisson approximation of $V_{f'(x')}$. The correction factor does not introduce correlation as it is a positive real number applied to each bin.

The resulted unfolded distribution $f(x)$ is shown in Figure 10 and the covariance matrix $V_{f(x)}$ in Figure 11.